

Model Evaluation and Their Interpretation

Aldebaro Klautau
Federal University of Pará (UFPA) / LASSE

Computational Intelligence Class

November 07, 2026

Figures of merit

Inform about misclassification error, confusion matrix, etc

Métricas de performance

m= N° de instâncias
Ex: casa analisadas

Equation 2-1. Root Mean Square Error (RMSE)

$$\text{RMSE}(\mathbf{X}, h) = \sqrt{\frac{1}{m} \sum_{i=1}^m \left(h(\mathbf{x}^{(i)}) - y^{(i)} \right)^2}$$

Métricas de performance

X= Valores dos atributos

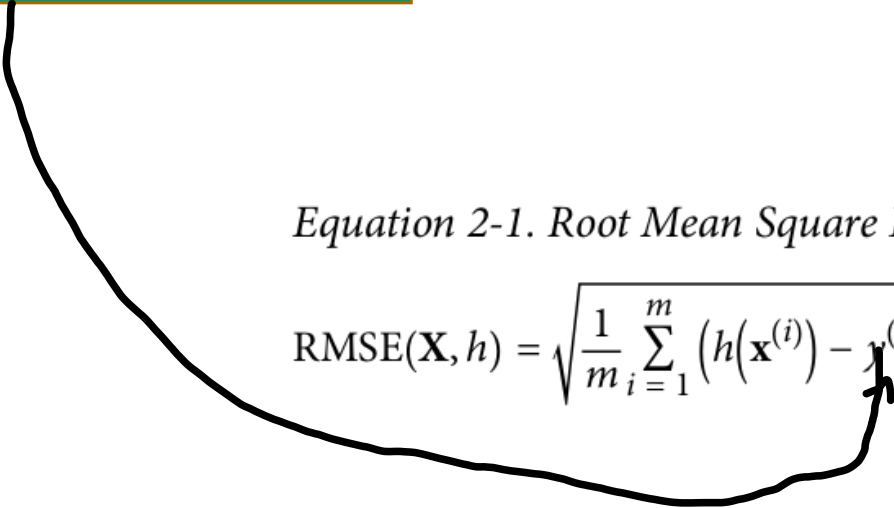
Equation 2-1. Root Mean Square Error (RMSE)

$$\text{RMSE}(\mathbf{X}, h) = \sqrt{\frac{1}{m} \sum_{i=1}^m \left(h(\mathbf{x}^{(i)}) - y^{(i)} \right)^2}$$

Métricas de performance

Y= Resultado
esperado

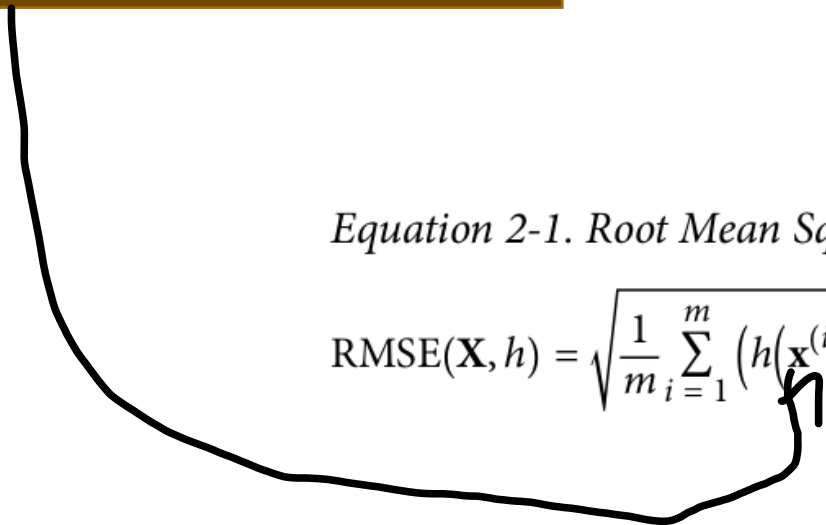
Equation 2-1. Root Mean Square Error (RMSE)

$$\text{RMSE}(\mathbf{X}, h) = \sqrt{\frac{1}{m} \sum_{i=1}^m \left(h(\mathbf{x}^{(i)}) - y^{(i)} \right)^2}$$
A thick, hand-drawn black arrow originates from the green box containing 'Y= Resultado esperado' and points towards the term $y^{(i)}$ in the RMSE equation.

Métricas de performance

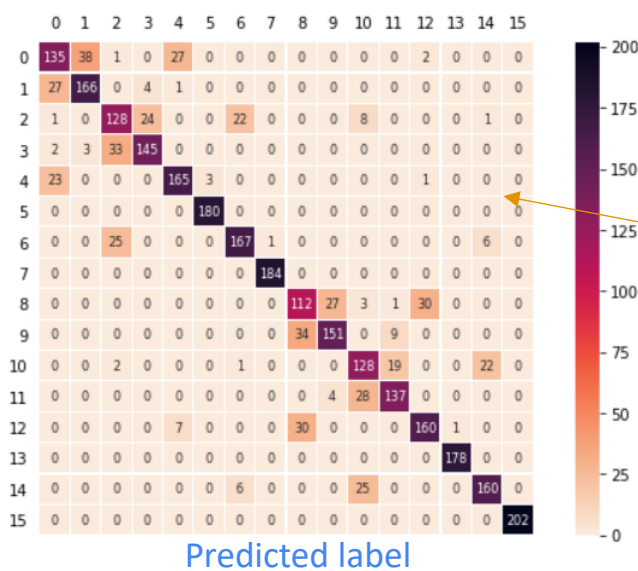
h = Função *machine learning*

Equation 2-1. Root Mean Square Error (RMSE)

$$\text{RMSE}(\mathbf{X}, h) = \sqrt{\frac{1}{m} \sum_{i=1}^m \left(h(\mathbf{x}^{(i)}) - y^{(i)} \right)^2}$$
A thick, hand-drawn black arrow originates from the text 'h = Função machine learning' and points directly to the $h(\mathbf{x}^{(i)})$ term in the RMSE equation.

Evaluation

True label



Confusion-matrix for classification

Generalization error: measure of how accurately an algorithm is able to predict outcome values for previously unseen data

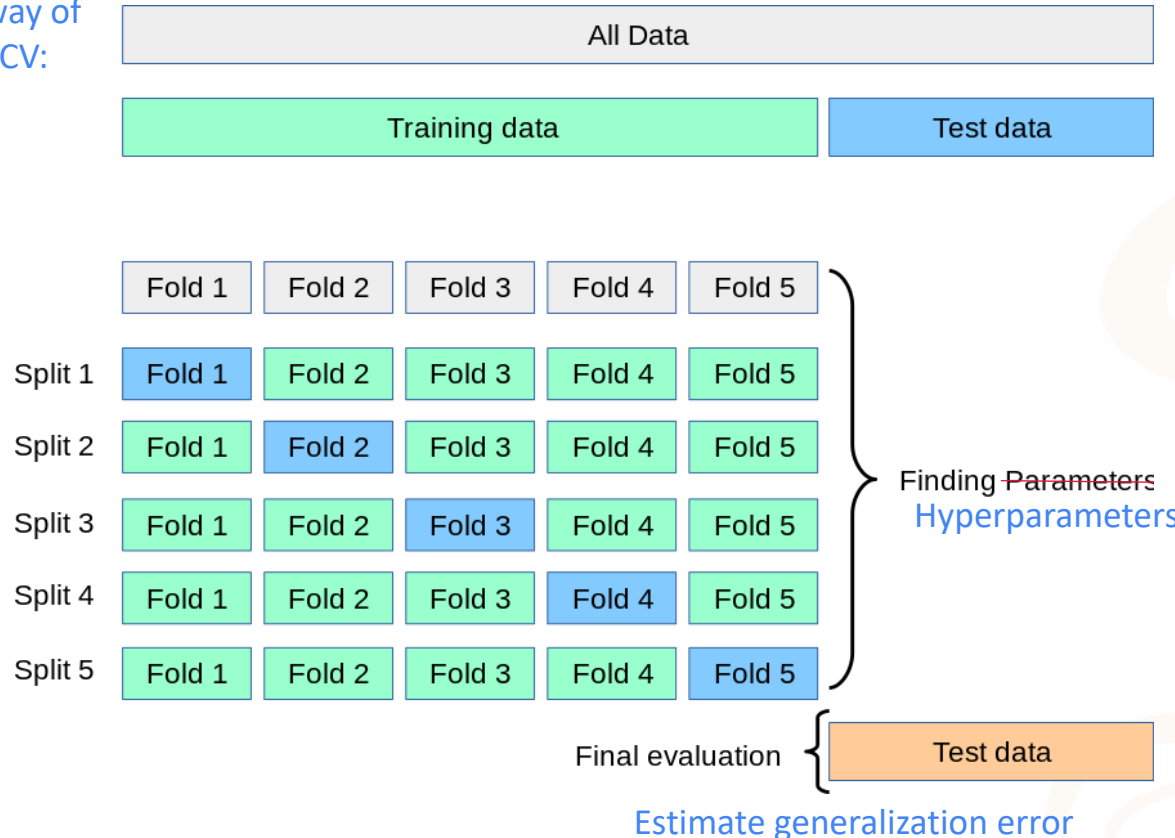
Generalization error estimates:

1. Using unseen **test dataset** (disjoint with the training set)
2. **Cross-validation** with N subsets (“splits” or “folds”)
3. **Leave-one-out:** cross-validation with a single example as the test set and all other examples composing the training set

Never estimate the generalization error using the training or validation sets!

Cross-validation (CV) for model selection and evaluation

One way of using CV:



CV can be used for estimating the generalization error and /or hyperparameters

CV can also use all data

Final model and its parameters should be obtained using all data

Estimate generalization error

CV by “hand”

Nested CV

Nested cross-validation was employed, with the inner loop used for model and hyperparameter selection and the outer loop used to obtain an unbiased estimate of the selected model's generalization performance.

- Outer-loop for estimating performance
- Inner-loop for hyperparameter tuning

Outer CV → estimate performance

- The **outer loop** separated the data into training and test folds.
- For each outer fold:
- Hold out the test group (in your case, using **grouped/LOSO-style splitting by participant**).
- Use only the remaining participants for model development.
- Train the final model selected within that training data.
- Evaluate it on the completely untouched outer test set.
- So the outer loop answers:
- **"How well will the entire model-selection procedure perform on a previously unseen patient?"**

Inner CV → model/hyperparameter selection

- Inside each outer training set, you performed CV to choose things such as:
- model/hyperparameters;
- potentially feature-selection parameters, depending on the experiment.
- So the inner loop answers:
- **"Given only the training patients, which model/configuration should I use?"**
- Then that selected configuration is retrained on the **entire outer-training set** and evaluated once on the outer test set.

Model selection with a validation set

Parameters

Values that are part of the deployed model,
e.g. neural network (NN) weights

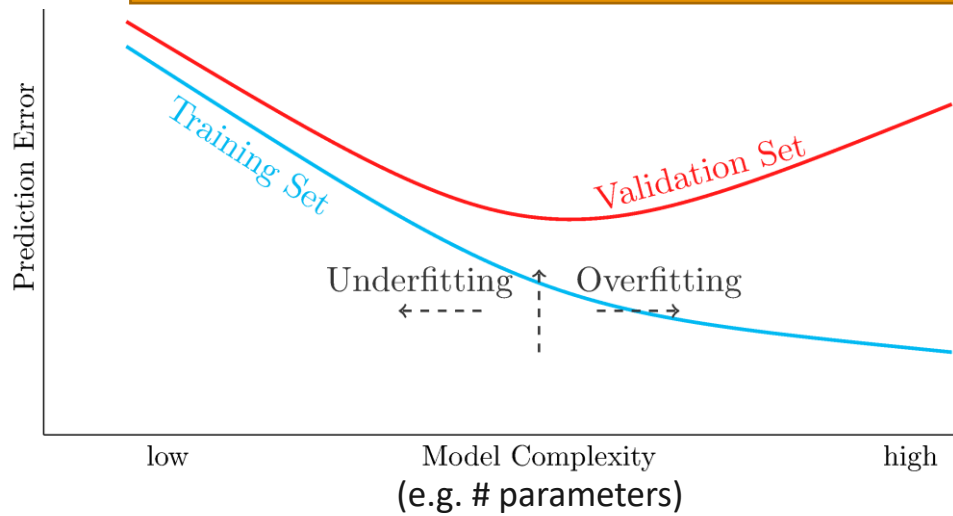


Hyperparameters

- Model hyperparameter: e.g, topology and size of a NN
- Algorithm hyperparameters: learning rate and mini-batch size

Model selection

Tune hyperparameters using a validation set to avoid overfitting and underfitting



Modify hyperparameters that influence the model complexity and adopt the values that provide good performance on validation set

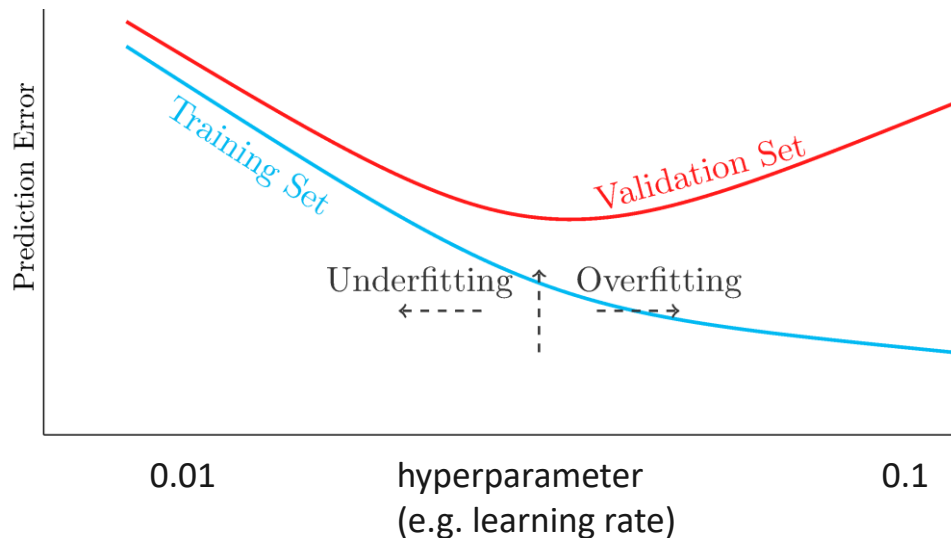
This strategy is also used to find the best value for a given hyperparameter

Model selection with a validation set (2)

DL models have many hyperparameters!

Model selection

Tune hyperparameters using a validation set to ~~avoid overfitting and underfitting~~ find their best values



Some frameworks for automatic model selection:

Scikit-learn

https://scikit-learn.org/stable/model_selection.html

Optuna

<https://optuna.org>

KerasTuner

https://keras.io/keras_tuner

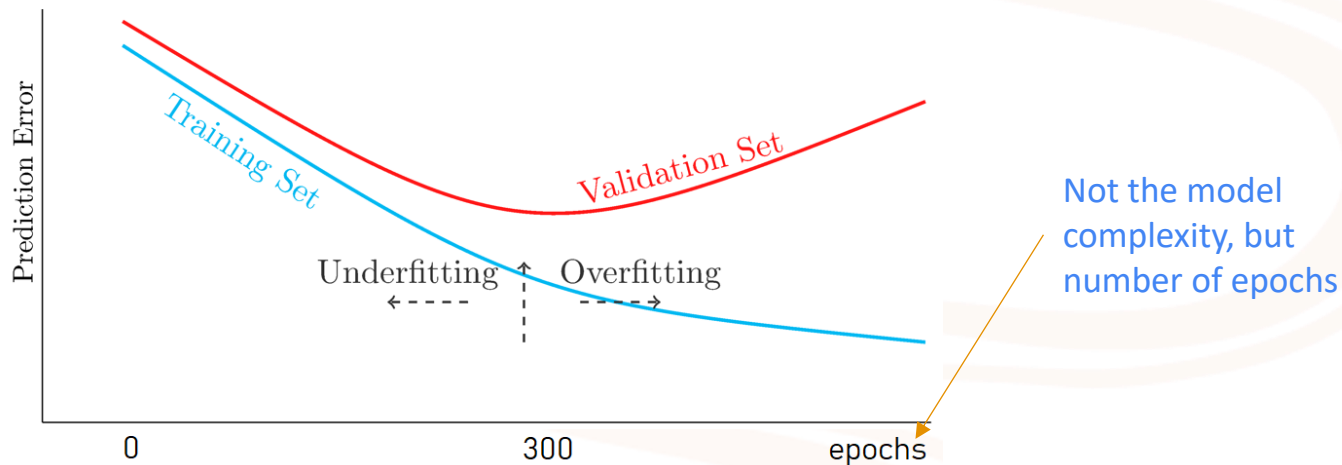
Early stop with a validation set

Assume model selection has indicated a set of hyperparameters

For how long should the model be trained? For how many epochs?

Epoch: one complete cycle over all training set examples that may have been shuffled

“Early stop”: interrupt training before maximum number of epochs and keep model that maximizes performance on validation set

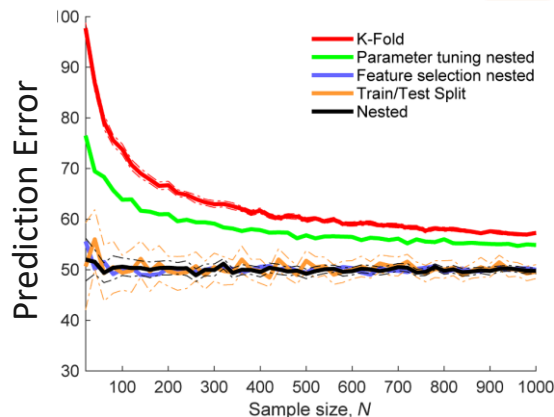
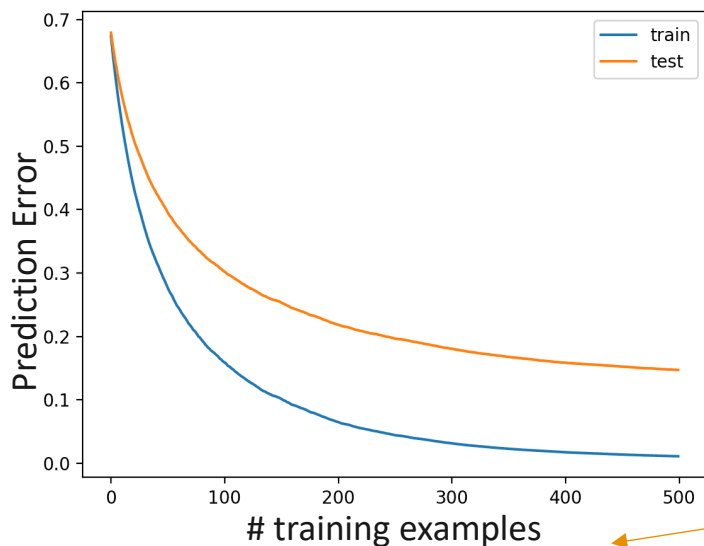


Sample complexity

Given a set of hyperparameters and a model trained with N epochs

What is the minimum number of training examples for good performance?

The sample complexity depends on the model, hyperparameters, data, learning algorithm, etc.



[1] Machine learning algorithm validation with a limited sample size", A. Vabalas et al, 2019

Table 1. The comparison on sample complexity to attain ϵ approximate point of stability.

Algorithm	Sample complexity
REINFORCE ¹³	$\mathcal{O}(1/\epsilon^2)$
PGT ¹⁴	$\mathcal{O}(1/\epsilon^2)$
GPOMDP ¹⁵	$\mathcal{O}(1/\epsilon^2)$
SVRPG ⁴	$\mathcal{O}(1/\epsilon^{5/3})$
HAPG ²²	$\mathcal{O}(1/\epsilon^3)$
IS-MBPG ¹⁸	$\mathcal{O}(1/\epsilon^3)$
DP-RBPG (this study)	$\mathcal{O}(1/\epsilon^3)$

[2] A randomized block policy gradient algorithm with differential privacy in Content Centric Networks, L. Wang et al, 2021.

Not the model complexity, nor the number of epochs, but the number of training examples

Interpretation and visualizing what is going on



Another dataset: Iris

Iris has three classes with 50 examples of each class, and four input features: width and length for petal and sepal

iris setosa



petal

sepal

iris versicolor



petal

sepal

iris virginica

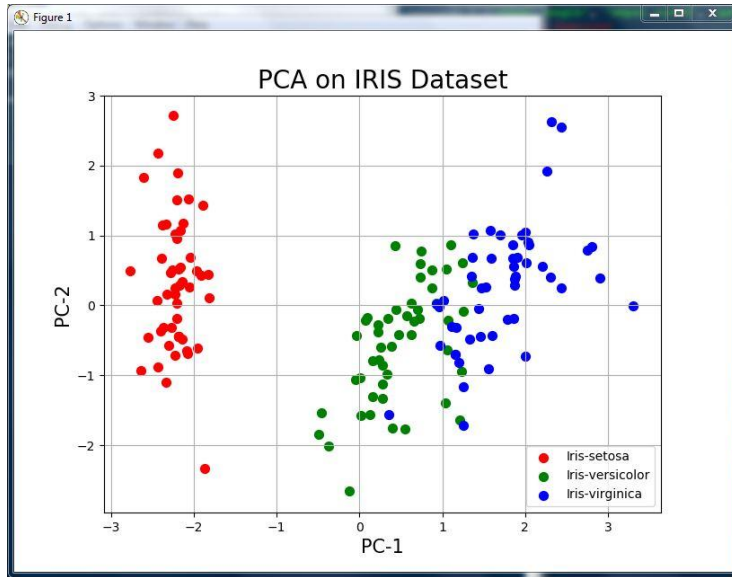


petal

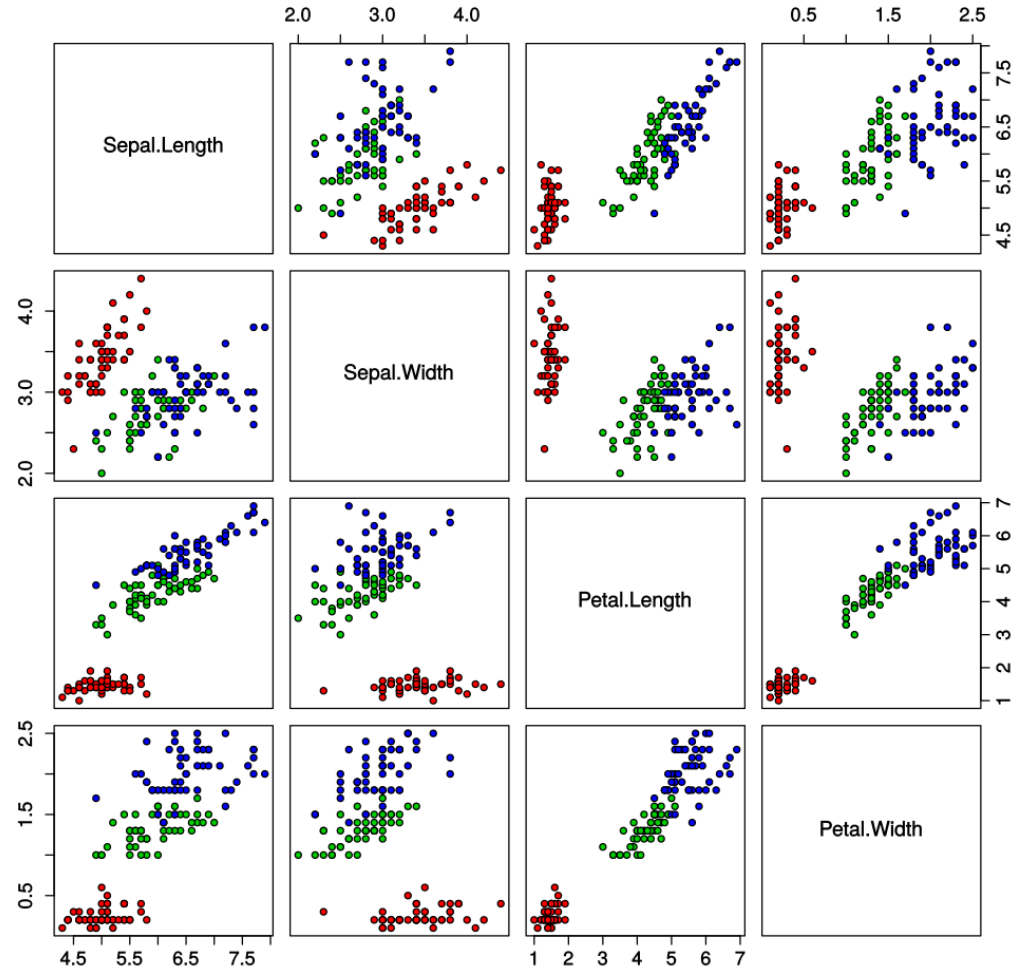
sepal

Iris

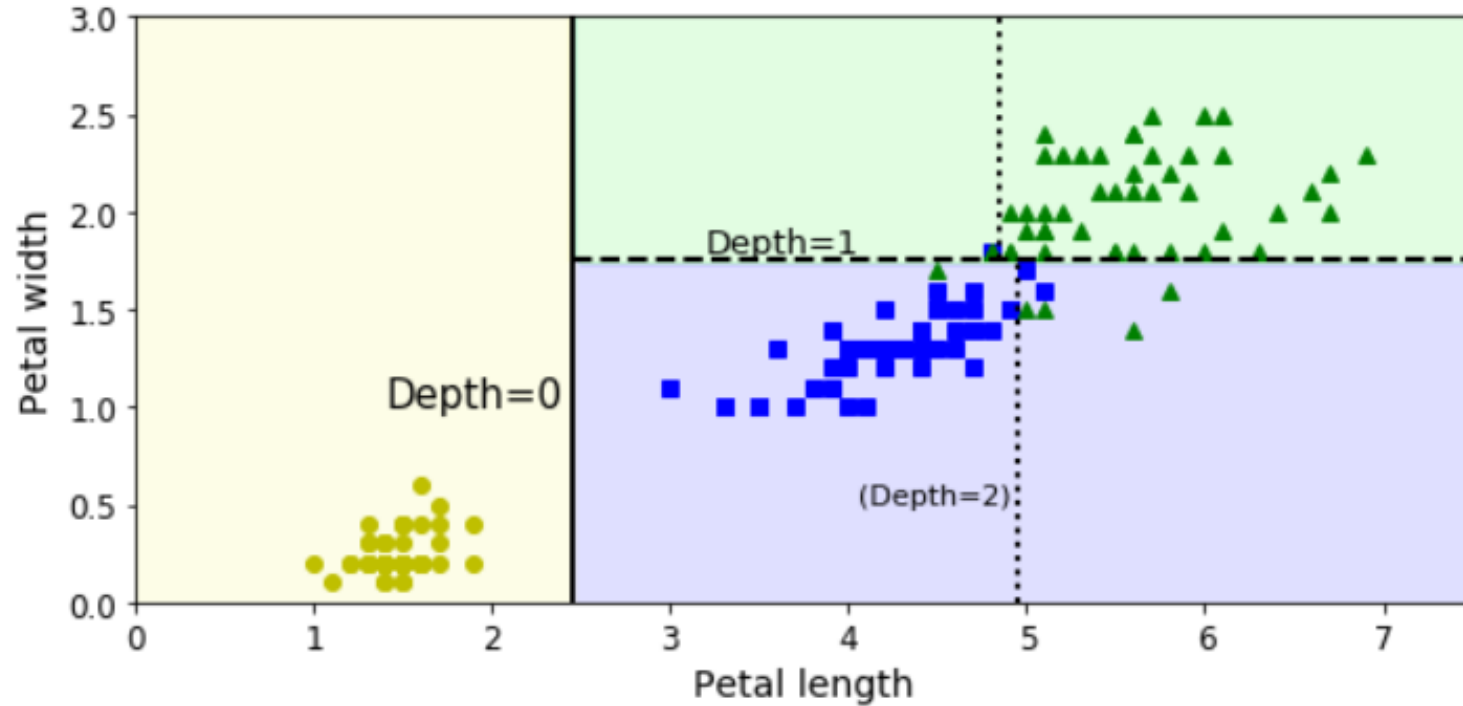
Principal component analysis (PCA)
for dimensionality reduction:



Iris Data (red=setosa,green=versicolor,blue=virginica)



Exemplo: Dataset Iris



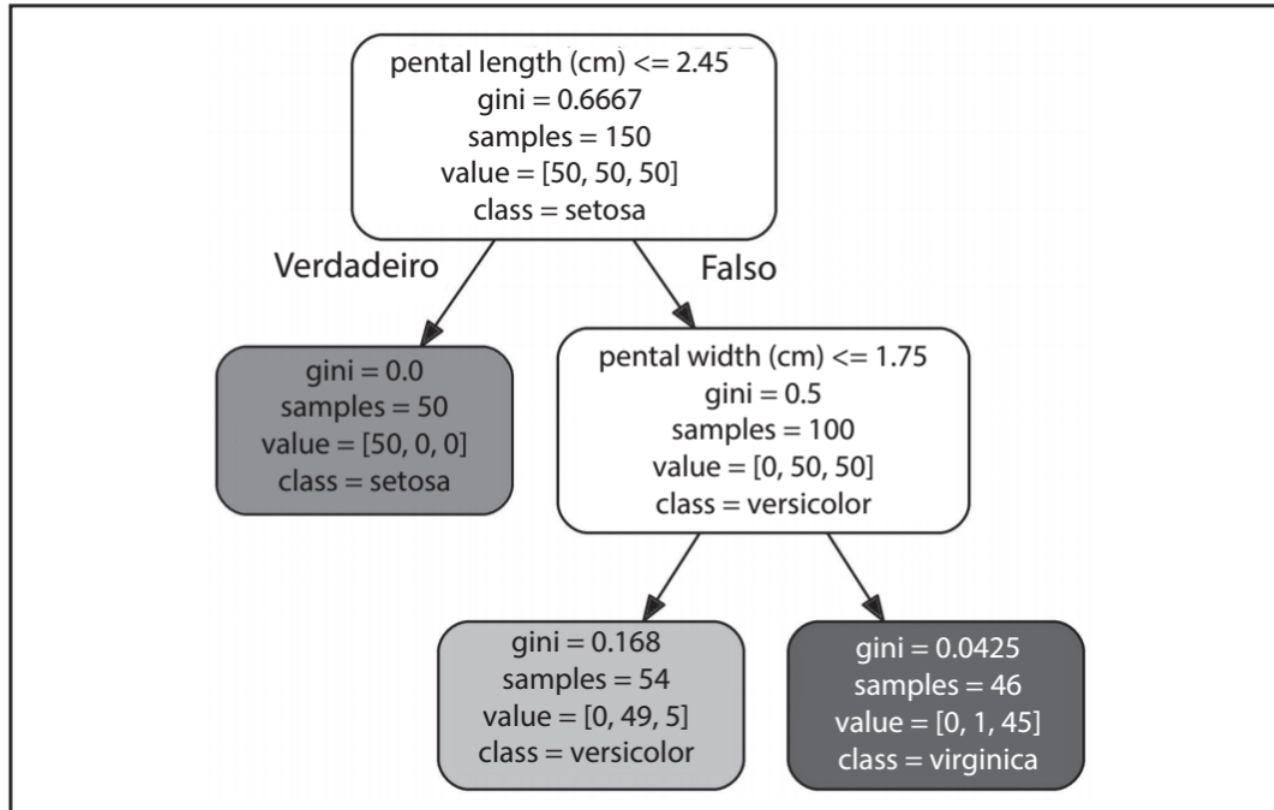
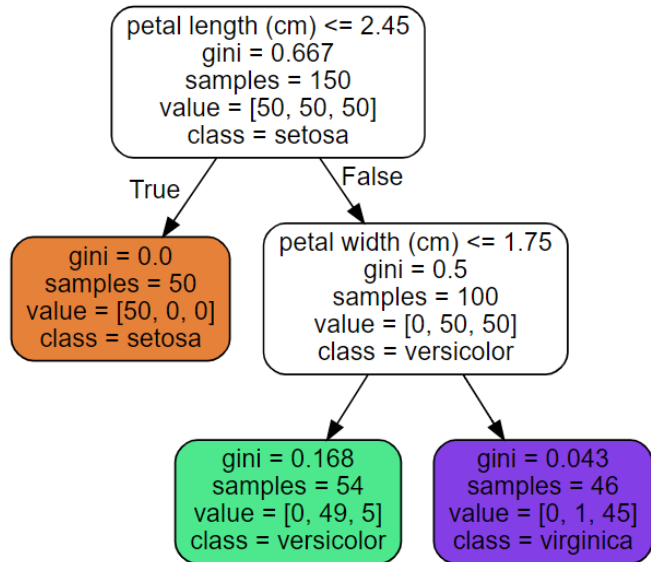
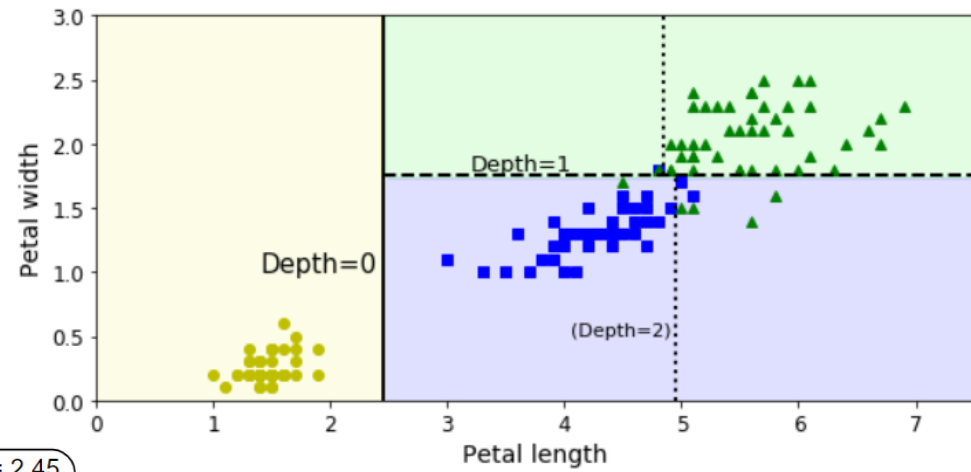


Figura 6-1. Árvore de Decisão da íris



Experiment reproducibility is not only initializing RNG with a given seed

Three different
pseudo random
number
generators (RNG)
when simulating
with Tensorflow

```
1  import numpy as np
2  import tensorflow as tf
3  import random as python_random
4  # Start random number generation in
5  # well-defined initial state.
6  np.random.seed(123) # Numpy
7  python_random.seed(456) #Core Python
8  tf.random.set_seed(789) #Tensorflow
```

For reproducibility, one needs also to properly provide dataset and code